License Plate Recognition Processing Using DSP Features

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

in

Electronics and Communication Engineering

of

APJ Abdul Kalam Technological University

by

MOHAMED FIZZA ALI (VAS24EC042)
VAISHNAV SUDHEER MULLAPILLY (VAS24EC061)
ADITHYA N S (VAS24EC008)
ADARSH P R (VAS24EC004)

(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Department of
Electronics and Communication Engineering

Vidya Academy of Science & Technology
Thalakkottukara, Thrissur - 680 501
(http://www.vidyaacademy.ac.in)

Oct 2026

Department of Electronics and Communication Engineering
Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680 501

(http://www.vidyaacademy.ac.in)

(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Certificate

This is to certify that the report entitled **"LICENSE PLATE RECOGNITION PROCESSING USING DSP FEATURES"** Submitted by **MOHAMED FIZZA ALI (VAS24EC042), VAISHNAV SUDHEER MULLAPILLY (VAS24EC061), ADITHYA N S (VAS24EC008), ADARSH P R (VAS24EC004)** of Vidya Academy of Science And Technology, Thalakkottukara, Thrissur - 680 501 in partial fulfillment of the requirements for the award of **Degree of Bachelor of Technology in Electronics And Communication** of **APJ Abdul Kalam Technological University**, during the academic year 2026-2027. The Project report has been approved as it satisfies the academic requirements in the respect of mini project work prescribed for the said degree.

Ms. Fetsy K Francis
(Project Supervisor)
Assistant Professor
Dept.of ECE
Vidya Academy of Science and Technology, Thrissur

Ms. Ashitha P R
(Project Co-supervisor)
Assistant Professor
Dept.of ECE
Vidya Academy of Science and Technology, Thrissur

Dr. Premanand B
Professor and HoD
Prof.& Head Dept.of ECE
Vidya Academy of Science and Technology
Thrissur

UNDERTAKING

We, the undersigned, **MOHAMED FIZZA ALI (VAS24EC042), VAISHNAV SUDHEER MULLAPILLY (VAS24EC061), ADITHYA N S (VAS24EC008), and ADARSH P R (VAS24EC004)**, hereby undertake that the mini project work entitled "**License Plate Recognition Processing Using DSP Features**", is carried out by ourselves under the valuable guidance of **Ms. Fetsy K Francis**, Associate Professor, Electronics and Communication Engineering Department, Vidya Academy of Science and Technology, Thalakkottukara, Thrissur, in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology in Electronics And Communication** of **APJ Abdul Kalam Technological University,** during the academic year 2026-2027.

MOHAMED FIZZA ALI
(VAS24EC042)
VAISHNAV SUDHEER
MULLAPILLY (VAS24EC061)
ADITHYA N S (VAS24EC008)
ADARSH P R (VAS24EC004)

Thrissur
Oct 2026

Acknowledgement

During the course of our project work several persons collaborated directly and indirectly with us. Without their support it would be impossible for us to finish our work. That is why we wish to dedicate this section to recognize their support.

We want to start expressing our thanks to our project supervisor **Ms. Fetsy K Francis**, Assi. Prof, & project co-supervisor **Ms. Ashitha P R**, Assi. Prof., Dept.of ECE Dept. of Electronics and Communication, because of their valuable advice and guidance towards this work. We received motivation, encouragement and hold up from them during the course of work.

We are grateful to express our thanks to all the faculty members of our department for their support. We articulate our gratitude to all our friends for their support and help for this work.

We are thankful to **Dr. Premanand B**, Head of Electronics and Communication Engineering Department, and our Principal **Dr. Sunitha C**, for their sole co-operation.

Last, but not the least we wish to express our gratitude to God Almighty for His abundant blessings without which this effort would not have been successful.

MOHAMED FIZZA ALI (VAS24EC042)
VAISHNAV SUDHEER MULLAPILLY (VAS24EC061)
ADITHYA N S (VAS24EC008)
ADARSH P R (VAS24EC004)

Fifth Semester (2024 Admissions)
Vidya Academy of Science & Technology
Thrissur - 680 501.

Oct 2026

Abstract

This project presents the design and implementation of a vehicle license plate recognition system using digital image processing techniques. The primary objective is to investigate how image enhancement, edge detection, thresholding, and morphological operations can be applied to vehicle images to identify and extract license plate regions for character recognition. The system employs a Python-based processing backend using OpenCV, where input images undergo preprocessing operations such as grayscale conversion, contrast enhancement, noise reduction, gradient-based edge detection, and thresholding. Multiple candidate regions are generated and evaluated using their geometric and visual characteristics to identify potential license plate locations. The selected region is further enhanced through cropping, perspective correction, and image sharpening before being passed to an Optical Character Recognition (OCR) stage. RapidOCR is used as the principal recognition method, with additional recognition approaches available as alternatives. An Android application developed using Kotlin provides the user interface for capturing vehicle images, selecting the plate region, and communicating with the Python Flask backend. The implemented prototype demonstrates the application of digital image processing concepts in a practical vehicle recognition task. Experimental observations show that the system can recognize plate-like text under suitable image conditions, although incorrect readings and false candidate detections occur in challenging images. Since a labelled test dataset and systematic accuracy evaluation are not yet available, the overall recognition performance has not been quantitatively established. The project therefore demonstrates the practical role of digital signal processing in image-based recognition while identifying areas for further improvement.

Contents

CERTIFICATE

UNDERTAKING

ACKNOWLEDGEMENT **i**

ABSTRACT **ii**

LIST OF FIGURES **v**

LIST OF SYMBOLS AND ABBREVIATIONS **vii**

VISION, MISSION & PROGRAM EDUCATIONAL OBJECTIVES (PEO) **1**

PROGRAM OUTCOMES (PO) & PROGRAM SPECIFIC OUTCOMES (PSO) **3**

SUSTAINABLE DEVELOPMENT GOALS (SDG) **7**

SDGs,POs AND PSOs ADDRESSED WITH JUSTIFICATION **8**

1 INTRODUCTION **10**

1.1 Background 10

1.2 Problem Statement 10

1.3 Objectives 11

1.4 Scope 11

1.5 Organization of the Report 12

2 LITERATURE REVIEW AND THEORETICAL BACKGROUND 13

2.1 ALPR Processing Stages 13
2.2 Digital Image Representation 13
2.3 Spatial Filtering and Convolution 14
2.4 Local Contrast Enhancement 15
2.5 Spatial Derivatives and Edges 15
2.6 Thresholding 16
2.7 Mathematical Morphology and Components 16
2.8 Resampling and Perspective Mapping 17
2.9 OCR as a Separate Recognition Stage 17

3 DSP METHODOLOGY AND IMAGE-PROCESSING PIPELINE 18

3.1 System Architecture 18
3.2 Processing Workflow 18
3.3 Image Input and Scale Normalization 19
3.4 Grayscale Conversion, CLAHE, and Denoising 19
3.5 Gradient, Edge, and Threshold Proposals 20
3.6 Candidate Regions and Heuristic Scoring 20
3.7 Candidate Crop and Perspective Rectification 21
3.8 Plate Enhancement and OCR Variants 21
3.9 Recognition and Format Ranking 21

4 SOFTWARE IMPLEMENTATION AND ANDROID–FLASK INTEGRATION 23

4.1 Python Pipeline Modules 23
4.2 Flask Image Endpoint 23
4.3 Android Capture and Manual Selection 24
4.4 Request Scheduling and Response Handling 24
4.5 Separation of DSP and Application Responsibilities 25

5 RESULTS AND DISCUSSION 26

5.1 Available Evidence 26
5.2 Observed Processing Stages 27
5.3 Recorded Output 27

5.4 Discussion and Limitations 29
5.5 Recommended Quantitative Evaluation 30

6 CONCLUSION AND FUTURE SCOPE 31
6.1 Conclusion 31
6.2 Future Scope 32

REFERENCES 32

7 APPENDIX 35
7.1 Localization Preprocessing Excerpt 35
7.2 Candidate Scoring Excerpt 36
7.3 Plate Enhancement Excerpt 37
7.4 OCR Selection Excerpt 38
7.5 Flask Upload Excerpt 40

List of Figures

3.1 Source-verified Android, Flask, and ALPR processing architecture. . . . 18
3.2 Image-processing and recognition workflow implemented by the backend. 19

5.1 Vehicle scene saved as the input to the investigation run. The plate is small relative to the road scene and the image includes rain and reflections. 26
5.2 Genuine intermediate outputs written by the backend investigation run. . 27
5.3 Saved candidate-proposal image. Boxes away from the plate illustrate false proposals that must be rejected by later crop recognition and ranking. 28
5.4 Selected crop and OCR-prepared image from the saved run. 28
5.5 Saved annotated output showing the selected plate box and the returned text. 29

LIST OF SYMBOLS AND ABBREVIATIONS

ALPR Automatic License Plate Recognition

API Application Programming Interface

CLAHE Contrast Limited Adaptive Histogram Equalization

CV Computer Vision

DSP Digital Signal Processing

OCR Optical Character Recognition

ROI Region of Interest

Sobel G_x Horizontal spatial intensity derivative

$I(x, y)$ Image intensity at pixel location (x, y)

Vision of the Institution

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

Mission of the Institution

M1: Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.

M2: Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.

M3: Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Vision of the Department

To be a centre of excellence in electronics and communication engineering, developing innovative and ethical engineers for sustainable technological development.

Mission of the Department

M1: Impart quality education with a strong knowledge foundation in core concepts, and complex problem-solving skills in electronics and communication engineering.

M2: Facilitate research, innovation, and industry-oriented practices in emerging areas of electronics, communication, and embedded technologies, fostering sustainable engineering solutions.

M3: Develop professional graduates with leadership, ethical values, teamwork, and lifelong learning abilities, with a commitment to societal well-being.

Program Educational Objectives (PEOs)

PEO-1: Graduates will apply strong technical competencies and problem- solving abilities in electronics and communication engineering to develop sustainable solutions to complex real-world challenges.

PEO-2: Graduates will excel in professional practice as innovators, entrepreneurs, educators, and researchers, applying interdisciplinary knowledge and skills in their careers.

PEO-3: Graduates will demonstrate professional ethics, teamwork, and leadership qualities, contributing responsibly to society and sustainable technological development.

Program Outcomes (POs)

PO1	**Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	**Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	**Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/ develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	**Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8)
PO5	**Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	**The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, WK7)
PO7	**Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8	**Individual and Collaborative Team Work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	**Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.
PO10	**Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	**Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Knowledge and Attitude Profile (WK)	
WK1	A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.
WK2	Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.
WK3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.

Knowledge and Attitude Profile (WK) (Continued)	
WK4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK5	Knowledge, including efficient resource use, environmental impacts, whole-life cost, re-use of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.
WK6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
WK7	Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.
WK8	Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.
WK9	Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Program Specific Outcomes (PSOs)

PSO-1	Analyze and design electronic circuits, VLSI systems, and embedded architectures to develop efficient hardware solutions for engineering applications.

PSO-2	Solve real-world electronics and communication engineering problems using appropriate hardware, software, and modern engineering tools while considering performance, safety, and sustainability.
PSO-3	Innovate in emerging areas of electronics and communication engineering while upholding professional ethics and addressing societal needs.

Sustainable Development Goals (SDGs)

SDG 1	**No Poverty:** End poverty in all its forms everywhere.
SDG 2	**Zero Hunger:** End hunger, achieve food security and improved nutrition and promote sustainable agriculture.
SDG 3	**Good Health and Well-being:** Ensure healthy lives and promote well-being for all at all ages.
SDG 4	**Quality Education:** Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.
SDG 5	**Gender Equality:** Achieve gender equality and empower all women and girls.
SDG 6	**Clean Water and Sanitation:** Ensure availability and sustainable management of water and sanitation for all.
SDG 7	**Affordable and Clean Energy:** Ensure access to affordable, reliable, sustainable and modern energy for all.
SDG 8	**Decent Work and Economic Growth:** Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.
SDG 9	**Industry, Innovation and Infrastructure:** Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.
SDG 11	**Sustainable Cities and Communities:** Make cities and human settlements inclusive, safe, resilient and sustainable.

SDGs addressed with justification

Project Title	SDGs addressed	Justification
License Plate Recognition Processing Using DSP Features	SDG 9: Industry, Innovation and Infrastructure	The student prototype applies digital image processing and software engineering methods to an automatic vehicle plate recognition task. No operational infrastructure or sustainability impact is claimed.

POs & PSOs addressed with Justification

Project Title	POs & PSOs addressed	Justification
License Plate Recognition Processing Using DSP Features	PO1 – Engineering Knowledge	Applies grayscale intensity, spatial filtering, gradients, thresholding and morphology to plate imagery.
	PO2 – Problem Analysis	Considers illumination, perspective, scale and false candidates in plate localization.
	PO3 – Design/Development of Solutions	Connects plate-candidate processing with OCR through an Android-to-Flask workflow.
	PO4 – Investigation of Complex Problems	Reviews saved intermediate images and one recorded run; no aggregate accuracy is claimed.
	PO5 – Modern Tool Usage	Uses OpenCV, Flask, RapidOCR, Kotlin and Android APIs.
	PO8 – Individual and Team Work	Lists the four group members and the integrated project components.
	PO9 – Communication	Documents implementation, available evidence and limitations.
	PO10 – Project Management	Records system design, source mapping and available qualitative evidence.
	PO11 – Life-long Learning	Connects DSP theory with practical computer-vision and OCR limitations.
	PSO2	Applies software tools to process images and exchange recognition results.

Chapter 1

INTRODUCTION

1.1 Background

Automatic license plate recognition (ALPR) converts a vehicle image into a textual registration string. A practical system must first find a small plate region within a larger scene, then prepare its pixels so that a recognizer can read the characters. Image capture, plate localization, image enhancement, and OCR are related stages, but they solve different problems. The present project is a hybrid prototype: classical OpenCV operations generate and score regions, while a general text recognizer supplies scene-text proposals and reads the enhanced crop.

This DSP project studies sampled-image intensity transformations. Grayscale conversion reduces three color channels to one intensity plane; local contrast enhancement changes the distribution of intensities; spatial filters suppress noise or emphasize transitions; thresholding produces binary masks; and morphology joins or cleans image structures. Contours, bounding boxes, candidate ranking, OCR, and Android networking use those transformed pixels, but are computer-vision, recognition, and application-integration tasks rather than digital filters themselves.

1.2 Problem Statement

A vehicle photograph may contain a small, distant, tilted, blurred, reflective, or partially occluded registration plate. Scene illumination can vary across the image, and road signs, vehicle trim, grille patterns, and reflections can resemble plate-like regions. A fixed

threshold or a single geometric rule can therefore miss a plate or produce false candidates. The project addresses this problem by combining several complementary image cues and by retaining intermediate outputs that can be inspected during development.

The prototype does not establish that an accepted result is correct. Its candidate scores are hand-designed heuristics, the OCR confidence is an engine output rather than a measured probability of correctness, and the registration-format test checks a broad character pattern. These distinctions guide the result discussion in Chapter 5.

1.3 Objectives

The objectives of the project are:

1. To inspect and document the actual image-processing and OCR pipeline implemented in the Python source.
2. To explain how grayscale conversion, contrast enhancement, filtering, spatial gradients, thresholding, morphology, and resampling support plate-region analysis.
3. To describe candidate generation, heuristic candidate scoring, crop enhancement, conditional perspective rectification, and OCR selection without presenting OCR as a DSP operation.
4. To document the Android-to-Flask data path and the roles of the Kotlin client and Python backend.
5. To evaluate only the saved visual and JSON evidence, report known failure modes, and avoid unsupported accuracy or speed claims.

1.4 Scope

The report covers the still-image ALPR prototype found in the project folder. It discusses the Python modules `localization.py`, `recognition.py`, `pipeline.py`, and `app.py`, the Flask route in `server.py`, and the Android capture, crop, and API code. It includes the available sample photographs and the saved processing stages in `alpr_output_curtain_investigation`. The report does not claim video tracking, a trained license-plate detector, a production service, automatic Android-side plate

localization, or a formal benchmark; those are not present in the inspected implementation or evidence.

1.5 Organization of the Report

Chapter 2 introduces digital image representation and reviews ALPR processing approaches. Chapter 3 explains the implemented DSP and image-processing stages and presents the system architecture and workflow. Chapter 4 documents the Python backend and Android integration. Chapter 5 discusses the genuine output evidence and limitations. Chapter 6 summarizes the work and lists future improvements. The appendix contains selected excerpts from the implementation rather than a copy of the full application.

Chapter 2

LITERATURE REVIEW AND THEORETICAL BACKGROUND

2.1 ALPR Processing Stages

ALPR systems commonly separate image capture, plate localization, character reading, and result handling. Reviews of the field describe both classical pipelines based on edges, morphology, connected components, and character segmentation and learned approaches based on object detectors and neural OCR [2, 3]. These methods have different data, compute, and robustness requirements. The system inspected here is not an end-to-end trained ALPR model: it combines classical OpenCV proposals and heuristic ranking with a general OCR engine.

This separation is useful when interpreting results. A plate may be correctly localized but misread; OCR may read text in a sign that was not a plate; or a detector may select a vehicle feature before OCR is called. Therefore, a single returned string cannot identify which stage failed. The project records candidate boxes, per-candidate features, the selected OCR method, and intermediate image files to make some of those failure paths inspectable.

2.2 Digital Image Representation

A digital image is a spatially sampled and quantized signal. For a grayscale image, $I[m, n]$ denotes the stored intensity at row m and column n. In the common 8-bit repre-

sentation used by OpenCV, each grayscale sample ranges from 0 (black) to 255 (white). A color image contains three channel values at each coordinate; the project loads color input as BGR and converts it to a single-channel grayscale image for most localization and plate-preprocessing operations.

One conventional luminance approximation is

$$Y = 0.299R + 0.587G + 0.114B, \tag{2.1}$$

where R, G, and B are the red, green, and blue channel samples. Equation (2.1) explains the idea of intensity conversion; the application uses OpenCV's `COLOR_BGR2GRAY` conversion rather than implementing the coefficients directly. Reducing the image to one channel simplifies later thresholding and spatial filtering, but it discards color differences that could otherwise distinguish a plate from its surroundings.

Image size affects both detail and processing cost. The localization routine reduces an input whose longest side exceeds 1600 pixels using area interpolation. Downsampling can reduce computation but can remove fine character strokes on distant plates. The selected plate is later enlarged with cubic interpolation; interpolation creates additional samples but cannot recover scene information that was absent from the original pixels.

2.3 Spatial Filtering and Convolution

Many image operations can be described by a spatial kernel $h[i, j]$ applied to an input image $I[m, n]$:

$$O[m, n] = \sum_i \sum_j h[i, j] I[m - i, n - j]. \tag{2.2}$$

Low-pass kernels smooth rapid intensity variation and can suppress noise; derivative kernels emphasize spatial changes. The project applies Gaussian smoothing to candidate regions and during quadrilateral search, bilateral filtering to reduce noise while retaining edges, and Sobel differentiation to produce a horizontal-gradient map. Spatial filtering is not neutral: smoothing can weaken character boundaries, while sharpening can amplify noise or compression artifacts.

2.4 Local Contrast Enhancement

An image histogram counts how many pixels have each intensity. If useful plate strokes occupy a narrow part of the available range, contrast enhancement can make local differences easier for thresholding and OCR. The project applies Contrast Limited Adaptive Histogram Equalization (CLAHE) to grayscale images. CLAHE equalizes local tile histograms and clips excessive histogram peaks so that local contrast is improved without allowing one tile to dominate without limit. Its parameters differ by stage: localization uses an 8 by 8 tile grid, while plate OCR preprocessing uses a 4 by 4 grid. Contrast enhancement can reveal faint strokes, but it can also amplify rain, texture, and sensor noise; it does not restore lost detail [7].

2.5 Spatial Derivatives and Edges

An edge corresponds to a rapid change in intensity. A first-order spatial derivative estimates that change. In two dimensions, the gradient is

$$\nabla I = \begin{bmatrix} G_x \\ G_y \end{bmatrix}, \qquad |\nabla I| = \sqrt{G_x^2 + G_y^2}, \qquad \theta = \tan^{-1}\left(\frac{G_y}{G_x}\right), \tag{2.3}$$

where G_x and G_y are horizontal and vertical derivatives. A 3 by 3 Sobel kernel for the horizontal derivative is

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}. \tag{2.4}$$

The localization code computes G_x with `cv2.Sobel(..., 1, 0, ksize=3)` and thresholds its magnitude. This emphasizes vertical intensity transitions that often occur around character strokes and plate borders. It is one proposal cue, not a complete plate detector. Canny edge detection is also applied to the filtered image; it combines gradient estimation, non-maximum suppression, and threshold-based edge tracking. The implementation uses Canny thresholds 55 and 165 for localization, and a separate 35 and 120 setting when searching a candidate crop for a plate quadrilateral [4, 6].

2.6 Thresholding

Thresholding maps an intensity image into foreground and background classes. A binary threshold with value T may be written as

$$B[m,n] = \begin{cases} 255, & I[m,n] > T, \\ 0, & \text{otherwise.} \end{cases} \tag{2.5}$$

Otsu's method chooses a global threshold from the image histogram by seeking a split with low within-class variance. It is used on the Sobel-gradient response and in the candidate scorer's inverse threshold for dark components. Adaptive Gaussian thresholding instead computes a local threshold from a weighted neighborhood, conventionally $T[m,n] = \mu_G[m,n] - C$, where μ_G is a Gaussian-weighted local mean and C is a constant. The localization path uses a 21-pixel block and $C = 7$; plate preprocessing creates an additional 31-pixel adaptive variant with $C = 9$. Local thresholds can handle uneven illumination better than a single global value, but small block sizes can fragment strokes and large blocks can merge nearby structures [8].

2.7 Mathematical Morphology and Components

Morphological operations use a structuring element to change the shape of binary regions. Dilation expands foreground regions; erosion contracts them. Closing is dilation followed by erosion and can bridge small gaps, while opening is erosion followed by dilation and can remove small isolated components. The localization code applies horizontal rectangular closing to join edge and gradient responses, opening to clean one gradient map, and a small dilation to thicken the Canny candidate map. The OCR template fallback also applies a horizontal opening when separating character structures. The kernel shape and scale matter: a long horizontal element favors plate-like regions but can also merge unrelated scene edges [9].

Connected-component labeling groups neighboring foreground pixels and measures each component's bounding box and area. The project uses component dimensions to group aligned glyph-like regions and to compute a soft character-count feature. Contour extraction provides region boundaries and bounding rectangles for gradient, edge, and

threshold candidate masks. These are geometric computer-vision steps that consume processed images; they are not themselves OCR or a convolution filter.

2.8 Resampling and Perspective Mapping

Resizing changes the sampling grid. The project uses area interpolation when reducing a large scene and cubic interpolation when enlarging plate crops. If four plausible corners are found within a crop, a projective mapping can transform them to a rectangular destination. In homogeneous coordinates, the mapping is

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \tag{2.6}$$

where H is a 3 by 3 homography and λ is a scale factor. The code estimates H from four points and calls `warpPerspective`; if it cannot find a sufficiently plausible quadrilateral, it keeps the unwarped crop. Perspective mapping can normalize a slanted plate but cannot correct occlusion, severe blur, or incorrect corner estimates [10].

2.9 OCR as a Separate Recognition Stage

OCR maps image patterns to text labels. It may use learned features or explicit character templates, but it is not equivalent to the preceding DSP operations. RapidOCR is called for full-scene text proposals and enhanced plate variants. If RapidOCR does not take a high-confidence format fast path, available OCR choices can be ranked together with optional Tesseract and the project's OpenCV-rendered glyph-template fallback. The source imports `rapidocr_onnxruntime`; Tesseract requires both its executable and the optional Python binding. The template fallback is a local classifier, not a trained plate-recognition model [11, 12].

Chapter 3

DSP METHODOLOGY AND IMAGE-PROCESSING PIPELINE

3.1 System Architecture

Figure 3.1 summarizes the prototype as implemented. The Android application handles capture and manual plate-region selection. It sends the saved crop to the Flask service, which decodes the image and calls the Python ALPR pipeline. The returned JSON fields are stored and presented by the Android client. The figure is an architecture diagram derived from source inspection, not a screenshot of the running application.

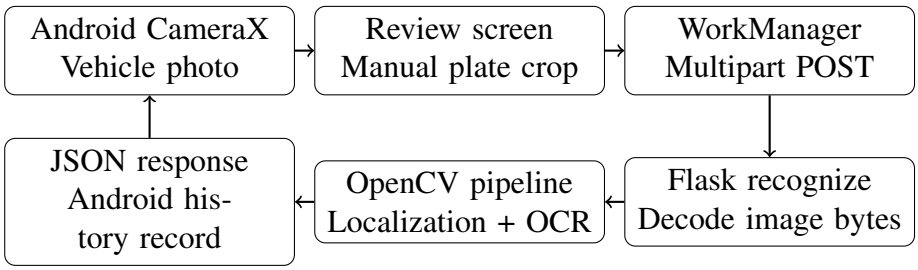


Figure 3.1: Source-verified Android, Flask, and ALPR processing architecture.

3.2 Processing Workflow

The backend accepts either an image path or a decoded BGR array. It reduces very large images, generates classical proposals, adds scene-text proposals when RapidOCR is available, crops and evaluates up to 15 candidates by default, and returns the best-

ranked text and box. Figure 3.2 presents the stages in order. Several branches are parallel proposal sources; the diagram groups them to keep the overall flow readable.

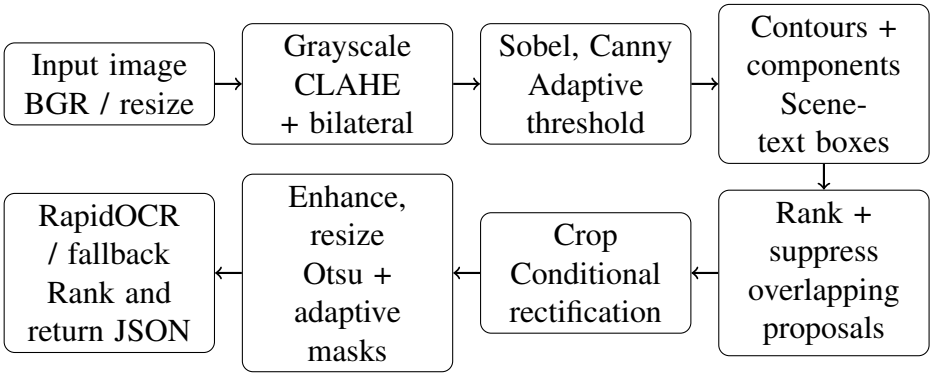


Figure 3.2: Image-processing and recognition workflow implemented by the backend.

3.3 Image Input and Scale Normalization

In `pipeline.py`, `recognize_image` loads color files with OpenCV or copies an already decoded image array. A grayscale array is converted back to BGR so later code has a consistent input shape. The working image is reduced only when its longest dimension exceeds 1600 pixels, using `INTER_AREA`. The project keeps the original dimensions for the final output box and scales box coordinates back to the source image when required.

3.4 Grayscale Conversion, CLAHE, and Denoising

The classical localization path converts BGR to grayscale, applies CLAHE with clip limit 2.2 and an 8 by 8 tile grid, then applies a bilateral filter with diameter 7 and color and spatial sigmas of 55. This order prepares the signal before derivative and binary operations. CLAHE can raise local contrast in a shadowed scene; the bilateral filter averages samples based on spatial proximity and intensity similarity, reducing some noise while limiting blur across sharp character edges. The visual output `02_gray_clahe.png` is a saved intermediate from the project, not a newly generated illustration.

3.5 Gradient, Edge, and Threshold Proposals

The localization code computes a 3 by 3 Sobel derivative in the horizontal direction, converts its magnitude to an 8-bit view, and applies Otsu thresholding to produce a gradient mask. It closes that mask with a scale-relative horizontal rectangular kernel and then performs a small opening. The Sobel operation emphasizes vertical transitions; the following morphology connects nearby responses into elongated regions that can be examined as plate candidates.

A second proposal map comes from Canny edges with thresholds 55 and 165. The code closes the edge map with the same horizontal kernel and dilates it once with a 3 by 3 rectangle. A third map comes from inverse adaptive Gaussian thresholding with a 21-pixel neighborhood and constant 7, followed by horizontal closing. Each mask can respond to different contrast and boundary conditions. The saved gradient and Canny maps are `02_gradient_x.png` and `02_edges.png`. The adaptive-threshold map is `02_adaptive_threshold.png`.

3.6 Candidate Regions and Heuristic Scoring

Contours from the gradient, edge, and adaptive-threshold masks are converted to bounding rectangles and passed to `_score_box`. A further path groups aligned character-sized connected components. The score rejects extreme or too-small boxes and combines six normalized features:

$$\begin{aligned} S_{loc} = {} & 0.27S_{aspect} + 0.27S_{brightness} + 0.16S_{contrast} \\ & + 0.18S_{components} + 0.07S_{rectangularity} + 0.05S_{edge}. \end{aligned} \tag{3.1}$$

The aspect term favors a broad one-line layout while allowing a near-square prior for a possible two-line layout. Brightness and dark-pixel fractions model a reflective light plate with darker glyphs; local standard deviation estimates contrast; component count supplies weak character evidence; contour fill estimates rectangularity; and a Canny mean supplies edge evidence. These are hand-selected rules, not learned coefficients or calibrated probabilities.

Overlapping proposals are suppressed by intersection-over-union (IoU): a new can-

didate is retained only when its IoU with each already selected box is below 0.68. If the classical locator returns no proposals, a bright-region contour fallback is attempted. The full-scene OCR path can add a candidate when detected text resembles a registration-like string and has plausible geometry. It can also introduce non-plate text, which is a known source of false proposals.

3.7 Candidate Crop and Perspective Rectification

The pipeline expands some tight glyph or OCR boxes to include nearby plate body, clips the box to the image, and extracts a crop. In `rectify_plate`, the crop is converted to grayscale, enhanced with CLAHE, lightly blurred, and passed through Canny with thresholds 35 and 120. A 3 by 3 close is applied before contour search. If a sufficiently large four-sided polygon is found, four ordered points are mapped to a rectangle by `getPerspectiveTransform` and `warpPerspective`; otherwise the crop is retained. Rectification is conditional, so the report does not claim that every crop is perspective-corrected.

3.8 Plate Enhancement and OCR Variants

The crop is converted to grayscale and processed with CLAHE (clip limit 2.5, 4 by 4 grid), a bilateral filter (diameter 5, sigmas 28 and 28), and an unsharp-mask-like weighted difference from a Gaussian-blurred version. It is resized with cubic interpolation to a target height selected from the crop size and padded by repeating border pixels. The code creates a global Otsu threshold, an adaptive Gaussian threshold (block size 31, constant 9), and their inverse masks. These variants let the OCR methods see different versions of the same sampled plate. They can make weak strokes easier to read, but interpolation cannot restore missing character detail and thresholding may merge or break glyphs.

3.9 Recognition and Format Ranking

RapidOCR is first called on grayscale, Otsu, and adaptive variants. A high-confidence string matching the project's Indian-format regular expression can use an early-return path. Otherwise, the implementation may also evaluate Tesseract (when both executable

and binding are available) and an OpenCV template reader, then rank returned strings with a format bonus. At pipeline level, the final candidate ordering gives priority to readings considered registration-like, then uses a combined candidate/OCR score. A registration-shaped string is therefore a ranking signal only: syntax cannot prove plate identity or official validity.

Chapter 4

SOFTWARE IMPLEMENTATION AND ANDROID–FLASK INTEGRATION

4.1 Python Pipeline Modules

The backend code is separated into small modules. `localization.py` defines candidate boxes, the heuristic box score, glyph-component grouping, mask generation, contour extraction, and crop creation. `recognition.py` contains the Indian-format expression, scene-text detection, plate-crop rectification, image variants, RapidOCR calls, optional Tesseract handling, and the OpenCV glyph-template reader. `pipeline.py` coordinates these components in `recognize_image`, ranks candidates, and can save diagnostics plus `results.json`. `app.py` adapts the core result to an application-facing status. The CLI entry point in `alpr/__main__.py` is separate from the Flask service.

4.2 Flask Image Endpoint

The Flask server in `server.py` exposes `POST /recognize`. It expects the uploaded file in the multipart field named `image`. The endpoint reads the bytes, converts them to a NumPy byte array, decodes a BGR image with `cv2.imdecode`, and returns an HTTP 400 JSON error if the field is missing or the image cannot be decoded. A valid image

is passed to `recognize_for_app`. The JSON response contains `plate_number`, `confidence`, `valid_indian_format`, `status`, and `error`. The development server listens on `0.0.0.0:5000` with debug mode disabled.

4.3 Android Capture and Manual Selection

The Kotlin application uses CameraX through `MainActivity.kt`. Its capture use case requests quality-oriented capture and saves the original vehicle image to private app storage. The review screen lets a user select the plate by dragging a rectangle or tapping two corners; the selection is stored as normalized coordinates so it remains meaningful when the displayed image is resized or rotated.

`ImageFiles.writeSelectionCrop` maps the normalized selection back to the source-image coordinate system, accounts for EXIF orientation, decodes only the chosen region, and saves a separate JPEG crop at quality 95. It uses a decode sample size so the crop's longest dimension does not exceed 1600 pixels. The original capture remains a distinct private file. This client therefore asks the user to localize the plate region; automatic candidate generation occurs in the Python backend if the backend receives a vehicle scene, but the app's normal request contains the manually selected crop.

4.4 Request Scheduling and Response Handling

After crop creation, the app saves a pending record and enqueues a persistent WorkManager request. The worker requires a connected network and uses exponential backoff beginning at 30 seconds. `RecognitionApi.kt` sends the crop as a multipart JPEG with field name `image` to the development endpoint configured in source as `http://192.168.137.1:5000/recognize`. The worker parses the JSON and updates record status, result text, confidence, and format-check information for the History screen. Failed and invalid-format records can be retried.

The interface crosses three boundaries: image storage on Android, HTTP transport, and image/OCR processing in Python. A request can fail before image processing because the server is unreachable, the upload is missing, or decoding fails. A successful response can still contain an incorrect OCR string. The hard-coded cleartext HTTP address

is a development configuration, not evidence of a production deployment; the application source itself recommends HTTPS for production.

4.5 Separation of DSP and Application Responsibilities

The Android app captures, displays, crops, stores, and submits images. Flask handles the transport endpoint and decodes uploaded bytes. OpenCV performs intensity transforms, spatial filtering, thresholding, morphology, contour operations, resampling, and geometric mapping. OCR engines perform text detection or recognition, and the format expression performs symbolic string checking. Keeping these responsibilities distinct makes it possible to discuss the DSP contribution without describing the project as an Android-only application.

Chapter 5

RESULTS AND DISCUSSION

5.1 Available Evidence

The project folder contains sample vehicle photographs and a saved investigation output directory. That directory preserves the input frame, grayscale/CLAHE image, gradient and edge maps, an adaptive-threshold map, candidate proposals, a plate crop, an OCR-prepared plate, an annotated detection, and a JSON result record. The figures in this chapter are copied from those genuine project artifacts. They show one saved processing run and are not a fabricated application screenshot or a controlled benchmark.



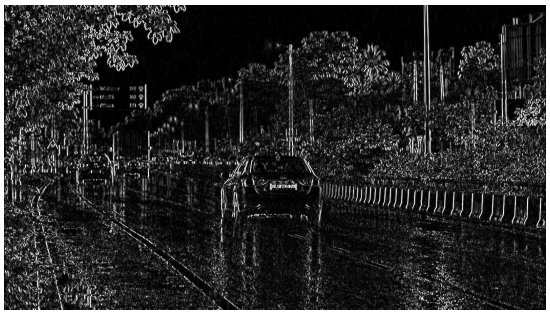
Figure 5.1: Vehicle scene saved as the input to the investigation run. The plate is small relative to the road scene and the image includes rain and reflections.

5.2 Observed Processing Stages

Figure 5.2 presents four saved diagnostic outputs from the same run. CLAHE produces a grayscale image used by the candidate pipeline. The Sobel output emphasizes horizontal-direction derivatives (vertical intensity transitions), Canny produces a thin edge map, and adaptive thresholding creates a local binary representation. These maps expose the trade-off between preserving character-level detail and producing connected regions suitable for candidate generation.



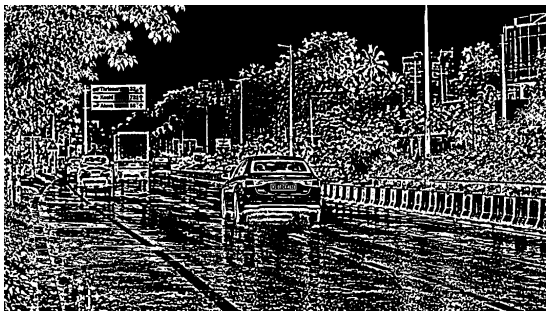
(a) Grayscale with CLAHE.



(b) Sobel G_x response.



(c) Canny edge map.



(d) Adaptive-threshold map.

Figure 5.2: Genuine intermediate outputs written by the backend investigation run.

The candidate map in Figure 5.3 shows multiple proposals on the vehicle, road sign, truck, and bright scene structures. Those boxes are not all license plates. This is direct visual evidence that classical geometry and intensity cues can produce false candidate regions in a busy scene. The top-ranked proposal for this saved run is the plate text region.

5.3 Recorded Output

The saved `results.json` records one selected result. Figure 5.5 shows the resulting overlay; Table 5.1 summarizes the fields that can be verified from the JSON. The



Figure 5.3: Saved candidate-proposal image. Boxes away from the plate illustrate false proposals that must be rejected by later crop recognition and ranking.



(a) Selected plate crop enlarged for inspection.



(b) Processed OCR input saved by the pipeline.

Figure 5.4: Selected crop and OCR-prepared image from the saved run.

displayed text visually agrees with the characters on the plate in Figure 5.1. This observation describes one example only; it is not an estimate of system accuracy.



Figure 5.5: Saved annotated output showing the selected plate box and the returned text.

Table 5.1: Fields recorded for the available investigation run.

Field	**Recorded value or interpretation**
Recognized string	KL07CX4823
Selected box	$(x, y, w, h) = (778, 520, 95, 27)$ in input-image pixels
Proposal source	`ocr-text`, generated from a full-scene OCR text proposal
OCR method	`rapidocr-otsu`
Format flag	True for the project's regular-expression format check
Evidence scope	One saved run; no labeled test-set summary or aggregate accuracy is present

5.4 Discussion and Limitations

The saved example shows that the hybrid proposal set can include the correct plate in a rainy, reflective scene and that the OCR-prepared crop can yield a readable string. At the same time, Figure 5.3 shows false candidate boxes over non-plate structures. The project team has also observed incorrect plate readings and false detections during prototype use; the available folder does not contain a labeled corpus of those failures that would support a rate calculation.

The localization score is a hand-tuned combination of geometric and pixel statistics, so a bright sign or grille may score well. Full-scene OCR can propose any sufficiently registration-like text, including signs. The crop-level perspective correction runs only when a sufficiently plausible four-sided contour is found. Small resolution, glare, motion blur, rain, occlusion, unusual character spacing, or steep perspective may destroy the information needed by OCR. The OpenCV glyph-template fallback uses rendered fonts rather than learned plate character data and is especially limited to clean, front-facing text.

The project also checks a broad regular-expression pattern. A true format flag means only that returned characters fit that pattern; it does not check an official registry, confirm that the text belongs to the photographed vehicle, or prove that OCR is correct. Likewise, the program's confidence fields are heuristic or OCR-engine outputs and have not been calibrated against manually labeled images. No overall accuracy percentage, test count, latency figure, or benchmark is reported because the inspected project materials do not support one.

5.5 Recommended Quantitative Evaluation

Future evaluation should first create a consented and manually labeled image set with ground-truth plate boxes and strings, then keep separate development and held-out test partitions. Useful measures would include plate-detection precision and recall at a stated intersection-over-union threshold, character-level accuracy, exact full-plate match rate, and processing latency on a named device. Each result should be grouped by conditions such as illumination, distance, blur, angle, rain, and occlusion. The measures should be computed over a declared sample count and accompanied by the evaluation protocol; none of these values is available for the present prototype.

Chapter 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project demonstrates a hybrid still-image plate-recognition workflow whose central engineering work is the preparation and analysis of image signals. The verified OpenCV path converts color input to grayscale, applies CLAHE and bilateral filtering, computes a horizontal Sobel response and Canny edges, thresholds intensity maps, applies morphology, extracts contours and connected components, scores candidate regions, and prepares plate crops for OCR. A possible quadrilateral is rectified by a perspective warp; otherwise the crop is retained. Multiple enhanced and thresholded views are passed to OCR. RapidOCR, optional Tesseract, and a local glyph-template fallback are recognition mechanisms that follow the DSP and computer-vision stages.

The Android client supplies a camera capture and manual crop-selection flow, while the Flask backend accepts the crop and returns a JSON result. The saved evidence includes one readable example and a candidate map with false proposals. The prototype's observed misreads and false detections, the heuristic candidate score, and the broad format check prevent any claim of measured overall accuracy. The results should therefore be interpreted as qualitative evidence of a working processing path rather than a validated performance guarantee.

6.2 Future Scope

Future work can improve the project in several measurable ways:

1. Build a labeled and permissioned evaluation set with varied vehicle types, plate layouts, lighting, distances, weather, and camera angles.

2. Record detection and recognition metrics with explicit definitions, a held-out test partition, and confidence calibration.

3. Compare the current heuristic candidate pipeline with a trained plate detector while retaining interpretable DSP preprocessing where useful.

4. Add controlled perspective and motion-blur handling, and test whether each enhancement improves recognition on the labeled set rather than assuming improvement.

5. Evaluate the optional OCR paths and local template fallback independently, recording the engine and preprocessing variant used for every output.

6. Replace the development HTTP endpoint with configurable HTTPS deployment before any production use, and define secure retention rules for captured vehicle images.

These are proposed improvements, not capabilities already implemented or measured in the current project.

Bibliography

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018, ISBN 978-0-13-335672-4.

[2] J. Shashirangana, H. Padmasiri, D. Meedeniya, and C. Perera, "Automated license plate recognition: a survey on methods and techniques," *IEEE Access*, vol. 9, pp. 11203-11225, 2021, doi: 10.1109/ACCESS.2020.3047929.

[3] P. Akpojotor et al., "Automatic license plate recognition on microprocessors and custom computing platforms: A review," *IET Image Processing*, vol. 15, no. 12, pp. 2717-2735, 2021, doi: 10.1049/ipr2.12262.

[4] OpenCV, "Sobel Derivatives," OpenCV 4.10.0 documentation. [Online]. Available: `https://docs.opencv.org/4.10.0/d2/d2c/tutorial_sobel_derivatives.html`. Accessed: Oct. 28, 2026.

[5] OpenCV, "Smoothing Images," OpenCV 4.11.0 documentation. [Online]. Available: `https://docs.opencv.org/4.11.0/d4/d13/tutorial_py_filtering.html`. Accessed: Oct. 28, 2026.

[6] OpenCV, "Canny Edge Detection," OpenCV 4.13.0 documentation. [Online]. Available: `https://docs.opencv.org/4.13.0/da/d22/tutorial_py_canny.html`. Accessed: Oct. 28, 2026.

[7] OpenCV, "Histograms - 2: Histogram Equalization," OpenCV 4.9.0 documentation. [Online]. Available: `https://docs.opencv.org/4.9.0/d5/daf/tutorial_py_histogram_equalization.html`. Accessed: Oct. 28, 2026.

[8] OpenCV, "Image Thresholding," OpenCV 4.12.0 documentation. [Online]. Available: https://docs.opencv.org/4.12.0/d7/d4d/tutorial_py_thresholding.html. Accessed: Oct. 28, 2026.

[9] OpenCV, "Morphological Transformations," OpenCV 4.13.0 documentation. [Online]. Available: https://docs.opencv.org/4.13.0/d9/d61/tutorial_py_morphological_ops.html. Accessed: Oct. 28, 2026.

[10] OpenCV, "Geometric Transformations of Images," OpenCV documentation. [Online]. Available: https://docs.opencv.org/doc/doxygen/html/da/d6e/tutorial_py_geometric_transformations.html. Accessed: Oct. 28, 2026.

[11] RapidAI, "RapidOCR," project repository and documentation. [Online]. Available: https://github.com/RapidAI/RapidOCR. Accessed: Oct. 28, 2026.

[12] Tesseract OCR, "Tesseract User Manual." [Online]. Available: https://tesseract-ocr.github.io/tessdoc/Home.html. Accessed: Oct. 28, 2026.

Chapter 7

APPENDIX

The following excerpts are selected from the application source for their DSP and integration value. They are copied excerpts, not standalone replacements for the application modules. The complete Android and Python projects remain in their original project folder.

7.1 Localization Preprocessing Excerpt

Listing 7.1 shows the verified grayscale, CLAHE, bilateral filtering, Sobel, Otsu, Canny, adaptive threshold, and morphology operations used to build candidate masks.

Listing 7.1: Localization preprocessing from `alpr/localization.py`

```
1     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
2     clahe = cv2.createCLAHE(clipLimit=2.2, tileGridSize=(8, 8)).apply(
      gray)
3     filtered = cv2.bilateralFilter(clahe, 7, 55, 55)
4
5     grad_x = cv2.Sobel(filtered, cv2.CV_32F, 1, 0, ksize=3)
6     grad_x = cv2.convertScaleAbs(grad_x)
7     _, grad_mask = cv2.threshold(grad_x, 0, 255, cv2.THRESH_BINARY |
      cv2.THRESH_OTSU)
8     horizontal_kernel = cv2.getStructuringElement(
9         cv2.MORPH_RECT, (max(9, int(width * 0.045)) | 1, max(3, int(
      height * 0.012)) | 1)
10    )
```

```
    gradient_map = cv2.morphologyEx(grad_mask, cv2.MORPH_CLOSE, horizontal_kernel, iterations=2)
    gradient_map = cv2.morphologyEx(
        gradient_map, cv2.MORPH_OPEN, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
    )

    edges = cv2.Canny(filtered, 55, 165)
    edge_map = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, horizontal_kernel, iterations=2)
    edge_map = cv2.dilate(edge_map, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)), iterations=1)

    adaptive = cv2.adaptiveThreshold(
        filtered, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV, 21, 7
    )
    adaptive_map = cv2.morphologyEx(adaptive, cv2.MORPH_CLOSE, horizontal_kernel, iterations=1)
```

7.2 Candidate Scoring Excerpt

Listing 7.2 contains the implemented weighted heuristic. The terms are ranking features and should not be interpreted as probability estimates.

Listing 7.2: Weighted candidate score from `alpr/localization.py`

```
    return float(
        0.27 * aspect_score
        + 0.27 * brightness_score
        + 0.16 * contrast_score
        + 0.18 * char_score
        + 0.07 * rect_score
        + 0.05 * edge_score
    )
```

7.3 Plate Enhancement Excerpt

Listing 7.3 creates the grayscale/CLAHE, denoised, sharpened, enlarged, Otsu, and adaptive variants used by the OCR stage.

Listing 7.3: OCR preprocessing from `alpr/recognition.py`

```
def preprocess_plate(crop: np.ndarray) -> dict[str, np.ndarray]:
    """Normalize contrast, upscale, and produce threshold variants for OCR."""
    if crop.ndim == 3:
        gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
    else:
        gray = crop.copy()
    if gray.size == 0:
        return {"gray": gray}
    # A light unsharp mask restores edges softened by a small source image.
    gray = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(4, 4)).apply(gray)
    smooth = cv2.bilateralFilter(gray, 5, 28, 28)
    sharpened = cv2.addWeighted(smooth, 1.7, cv2.GaussianBlur(smooth, (0, 0), 1.1), -0.7, 0)
    target_h = 112 if gray.shape[0] < 32 else 96
    scale = max(2.0, min(8.0, target_h / max(gray.shape[0], 1)))
    size = (max(1, int(gray.shape[1] * scale)), max(1, int(gray.shape[0] * scale)))
    enlarged = cv2.resize(sharpened, size, interpolation=cv2.INTER_CUBIC)
    enlarged = cv2.copyMakeBorder(enlarged, 8, 8, 10, 10, cv2.BORDER_REPLICATE)
    _, otsu = cv2.threshold(enlarged, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)
    adaptive = cv2.adaptiveThreshold(
        enlarged, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 31, 9
    )
    # Inverse masks are useful for component segmentation; standard black text
```

```
    # on white remains the preferred input to Tesseract.
    return {
        "gray": enlarged,
        "otsu": otsu,
        "adaptive": adaptive,
        "otsu_inverse": cv2.bitwise_not(otsu),
        "adaptive_inverse": cv2.bitwise_not(adaptive),
    }
```

7.4 OCR Selection Excerpt

Listing 7.4 shows the RapidOCR fast path and the conditional ranking of RapidOCR, optional Tesseract, and the local template reader.

Listing 7.4: OCR method selection from `alpr/recognition.py`

```
def recognize_crop(crop: np.ndarray, include_diagnostics: bool = False) -> OCRResult:
    """Recognize a plate crop with RapidOCR, Tesseract, or local templates."""
    rectified = rectify_plate(crop)
    variants = preprocess_plate(rectified)
    rapid = _rapidocr_read(variants)
    if rapid and rapid[0] >= 0.75 and INDIA_PATTERN.fullmatch(rapid[1]):
        processed = variants.get("otsu", variants.get("gray"))
        diagnostics = (
            {
                "selection_path": "rapidocr_format_fast_path",
                "ocr_method_candidates": [
                    {"method": rapid[2], "text": rapid[1], "confidence": rapid[0]}
                ],
                "template_evidence": {"evaluated": False, "reason": "RapidOCR fast path"},
            }
            if include_diagnostics
            else None
        )
```

```
        return OCRResult(rapid[1], rapid[0], rapid[2], processed, variants, diagnostics)
    if rapid and rapid[0] >= 0.50 and not shutil.which("tesseract"):
        processed = variants.get("otsu", variants.get("gray"))
        diagnostics = (
            {
                "selection_path": "rapidocr_no_tesseract_fast_path",
                "ocr_method_candidates": [
                    {"method": rapid[2], "text": rapid[1], "confidence": rapid[0]}
                ],
                "template_evidence": {"evaluated": False, "reason": "RapidOCR fast path"},
            }
            if include_diagnostics
            else None
        )
        return OCRResult(rapid[1], rapid[0], rapid[2], processed, variants, diagnostics)
    tess = _tesseract_read(variants)
    template_diagnostics: dict[str, object] | None = {} if include_diagnostics else None
    fallback_text, fallback_conf, fallback_method = _template_read(variants, template_diagnostics)
    choices: list[tuple[float, str, str]] = []
    if rapid:
        choices.append(rapid)
    if tess and tess[0]:
        choices.append((tess[1], tess[0], tess[2]))
    if fallback_text:
        choices.append((fallback_conf, fallback_text, fallback_method))
    if choices:
        # Prefer syntactically plausible reads when model confidences are
        # close, but keep OCR confidence as the dominant signal.
        choices.sort(key=lambda item: item[0] + (0.12 if INDIA_PATTERN.fullmatch(item[1]) else 0.0), reverse=True)
        confidence, text, method = choices[0]
```

```
        confidence = min(1.0, confidence)
    else:
        confidence, text, method = 0.0, "", "no-ocr-result"
```

7.5 Flask Upload Excerpt

Listing 7.5 shows how the HTTP endpoint receives the multipart image, decodes it, calls the application adapter, and forms a JSON response.

Listing 7.5: Flask image endpoint from `server.py`

```
@app.route("/recognize", methods=["POST"])
def recognize():
    if "image" not in request.files:
        return jsonify({"error": "No image uploaded"}), 400

    file = request.files["image"]
    image_bytes = file.read()

    image_array = np.frombuffer(image_bytes, dtype=np.uint8)
    image = cv2.imdecode(image_array, cv2.IMREAD_COLOR)

    if image is None:
        return jsonify({"error": "Invalid image"}), 400

    result = recognize_for_app(image)

    return jsonify({
        "plate_number": result.plate_number,
        "confidence": result.confidence,
        "valid_indian_format": result.valid_indian_format,
        "status": result.processing_status,
        "error": result.error,
    })
```



Department of Electronics and Communication

Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680 501

(http://www.vidyaacademy.ac.in)